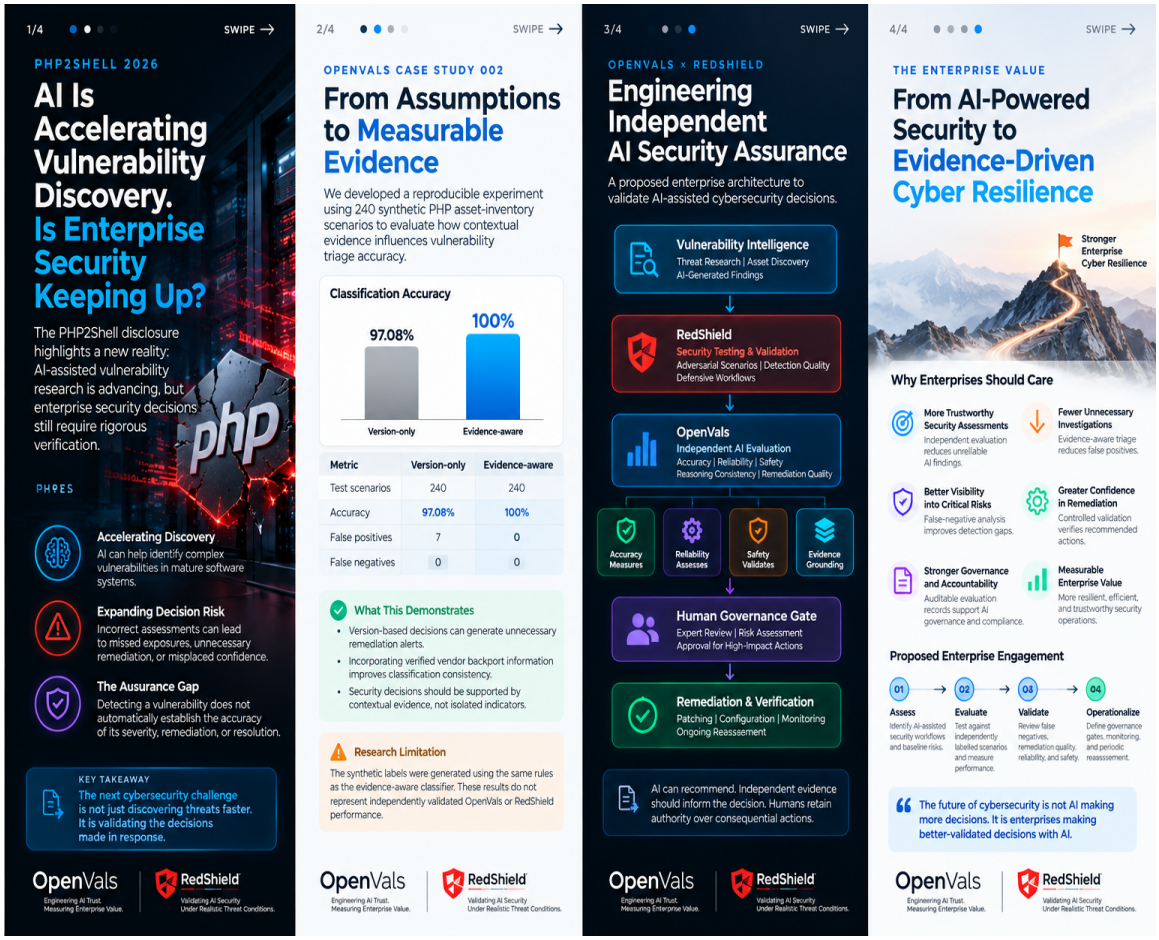
PHP2SHELL

Independent Assurance for AI-Assisted Vulnerability Triage

TECHNICAL WHITE PAPER | CASE STUDY 002 | PUBLIC RESEARCH EDITION | 23 OCTOBER 2026

OpenVals — Engineering AI Trust. Measuring Enterprise Value.
RedShield — Proposed defensive security validation companion.
Author: Vishwanath Akuthota --- Founder of Dr.Pinnacle, OpenVals and The Foundry's.



RESEARCH INTEGRITY NOTICE: Public incident facts and a synthetic deterministic rules benchmark are documented. No OpenVals/RedShield product performance, live exploit testing, or AI-model benchmark results are asserted.

Executive abstract

PHP2Shell denotes two critical vulnerabilities in PHP request handling disclosed on 21 October 2026: CVE-2026-109618 (out-of-memory cleanup corruption) and CVE-2026-109619 (stale request-owned global state). Under vulnerable conditions, unauthenticated remote code execution may be possible. The PHP project released fixes in PHP 8.2.35, 8.3.36, 8.4.27 and 8.5.12. [1–4]

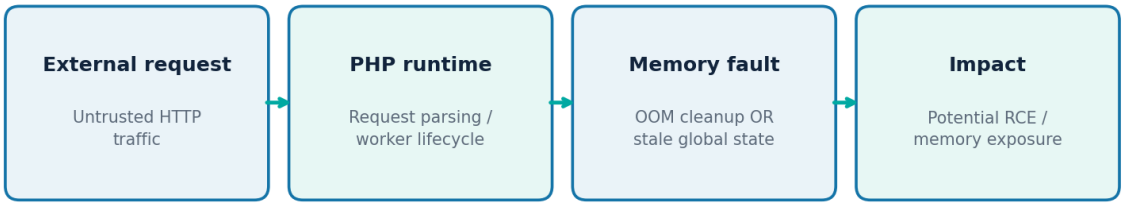
This white paper addresses a separate but consequential question: how should enterprises independently validate AI-generated vulnerability findings, triage decisions, and remediation advice? We distinguish public incident facts, a previously executed synthetic rules benchmark, and a proposed OpenVals × RedShield assurance design. No live exploitation, AI evaluation, or OpenVals/RedShield product execution is claimed.

1 | Incident anatomy and scope

The defects arise within PHP runtime internals rather than a particular application plugin. Exposure spans PHP installations serving HTTP requests, including mod_php and PHP-FPM. The PHP project reports particularly severe pre-application-code exposure in a demonstrated mod_php path. The precise exploitability of a given deployment depends on its runtime, configuration, and environment. [1–4]

CVE-2026-109618: an out-of-memory interruption may leave internal request structures incompletely initialized; later cleanup can act on invalid state. CVE-2026-109619: request-owned state may remain referenced after underlying memory is freed; a later request handled by the same worker can encounter stale state. These are high-level mechanisms, not exploit instructions. [1–3]

FIGURE 1 | PHP2Shell exposure path



Conceptual defensive view; no exploit instructions or payloads.

Figure: 01 Attack Surface

2 | Threat model and trust boundaries

Trust boundary A: internet traffic enters a reverse proxy or web server. Boundary B: PHP parses untrusted request data before or during application dispatch. Boundary C: worker memory and request lifecycle are expected to isolate requests. Boundary D: privileged remediation decisions cross from AI advice into operational action.

Key defensive assets are application availability, PHP worker integrity, credentials accessible to the service identity, downstream databases, and integrity of remediation records. Relevant failure modes include false-

negative vulnerability classification, unverified backport assertions, unsupported AI recommendations, and premature closure of an incident.

3 | Enterprise exposure and patch verification

Inventory runtime versions across hosts, containers, appliances, control planes and third-party bundles. Identify the actual PHP binary serving traffic, not merely a CLI version string. Record deployment mode, public exposure, package provenance, security bulletin and effective patched build. [1,4,5]

Apply official supported fixes immediately. For older branches, accept a vendor backport only with attributable bulletin, build provenance and installed-package verification. Restart affected PHP workers and confirm the fixed runtime is active. Temporary configuration mitigations should be treated as incomplete risk reduction, not remediation. [1,4,5]

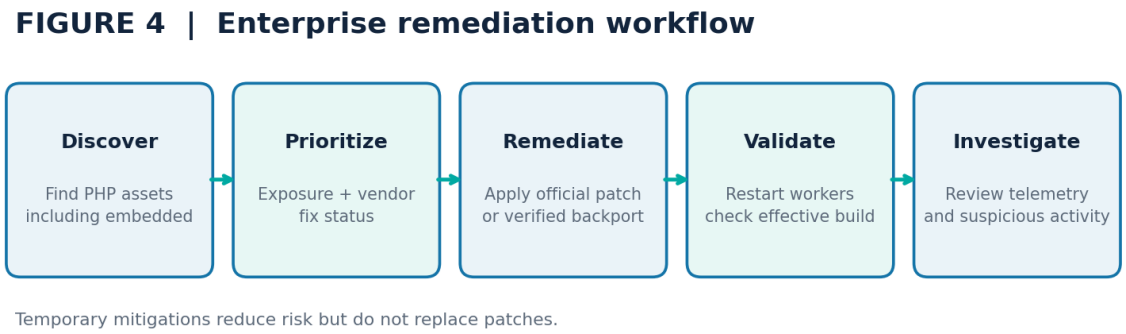


Figure: 04 Response Workflow

4 | Proposed OpenVals × RedShield technical design

The proposed design separates four functions: (i) source-of-truth asset and advisory ingestion; (ii) AI-generated analysis and remediation proposals; (iii) independent OpenVals evaluation against expert-labelled reference cases; and (iv) authorized RedShield defensive exercises using non-exploit scenarios. Human approval remains mandatory for production changes.

Evidence lineage should include advisory URL and retrieval time, package/build identifier, normalized affected-range rule, model identifier, prompt and rubric version, human reviewer, execution timestamp, and final disposition. The evaluation engine should never infer a backport solely from an optimistic model response.

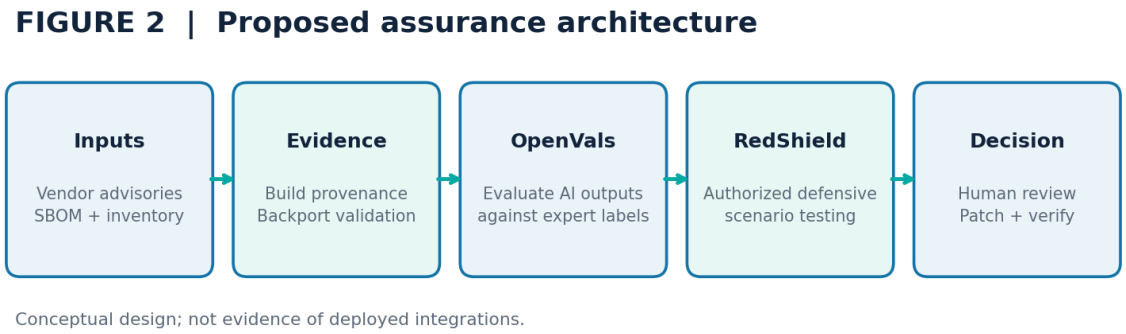


Figure: 02 Assurance Architecture

5 | Evaluation research questions and hypotheses

RQ1: Does evidence-aware classification reduce unnecessary remediation actions compared with version-only triage on independently labelled real-world inventories? RQ2: How often do AI systems fabricate or misinterpret vendor backport evidence? RQ3: Does independent evaluation reduce critical false negatives while preserving manageable analyst workload? RQ4: How robust are judgments under misleading advisory excerpts and ambiguous asset metadata?

Pre-register acceptance criteria with a security owner. Suggested guardrails: zero tolerated unreviewed high-severity false negatives in the release gate; explicit abstention on missing evidence; every recommended patch linked to a verifiable vendor bulletin; reproducible decision traces. These are governance proposals, not validated thresholds.

6 | Benchmark design and statistical protocol

Build a stratified benchmark across supported PHP branches, unsupported branches, vendor-patched backports, unknown build provenance, internet-facing and internal-only assets, contradictory advisories, and intentionally incomplete records. Obtain labels from at least two independent reviewers and adjudicate disagreements before evaluating model outputs.

Split development, calibration and locked holdout cases by vendor and asset family to reduce leakage. Report confusion matrices, precision, recall, false-negative rate, selective coverage, abstention rate, calibration, time-to-triage and reviewer minutes. Add uncertainty intervals and subgroup results; avoid claiming generalization from small synthetic samples.

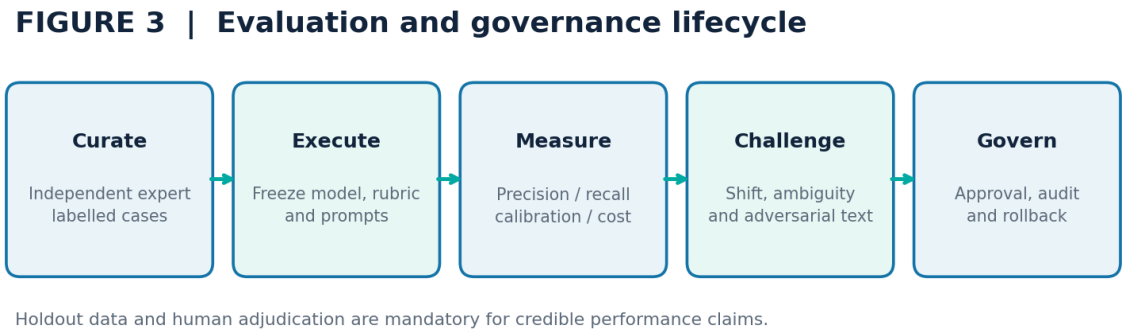


Figure: 03 Evaluation Pipeline

7 | Prior offline benchmark: what was actually measured

A previous local reproducible rules benchmark evaluated 240 synthetic PHP inventory scenarios (seed 20261023). Version-only classification produced TP=167, FP=7, FN=0, TN=66, accuracy=97.08%. Evidence-aware deterministic classification produced TP=167, FP=0, FN=0, TN=73, accuracy=100%. These values come from the supplied results.json and can be reproduced from the accompanying benchmark script.

CRITICAL LIMITATION: synthetic labels were generated with the same backport/version logic as the evidence-aware baseline. Its 100% accuracy is circular consistency, not independent predictive validity. Neither baseline is an LLM, and no PHP exploit, OpenVals SDK, RedShield platform, production telemetry, or enterprise deployment was tested.

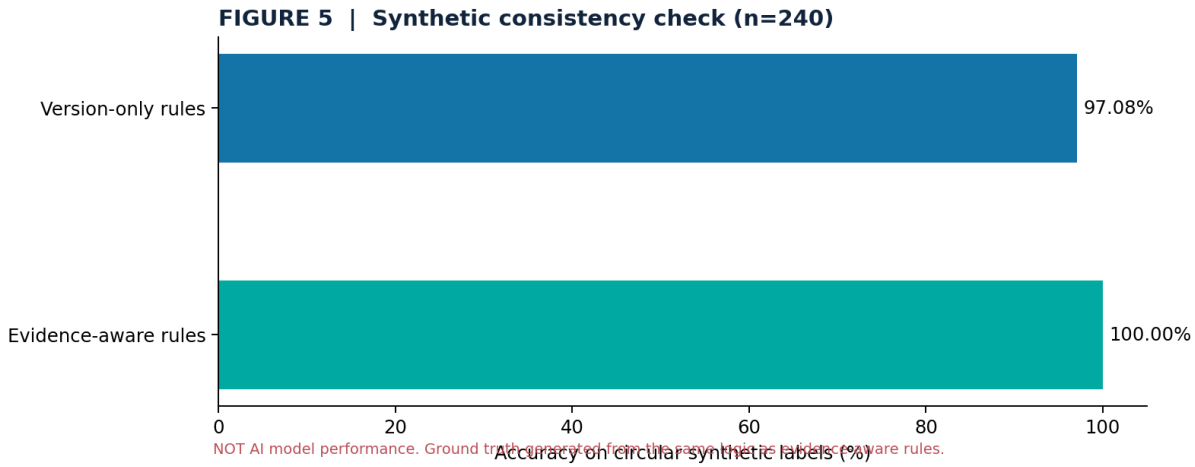


Figure: 05 Synthetic Results

Metric	Version-only	Evidence-aware
Synthetic scenarios	240	240
Accuracy	97.08%	100%
False positives	7	0
False negatives	0	0

8 | Security and governance controls

Require signed or traceable evidence for vendor patch claims. Enforce least-privilege access to asset inventories, separation of AI recommendation from patch execution, human approval for production changes, immutable evaluation traces and incident-response escalation on suspicious host activity.

Test prompt injection against retrieved advisory text as an untrusted input: the AI must not accept embedded instructions to change its role, conceal uncertainty, or bypass approval. Defensive test cases should use harmless simulated evidence, never live malicious payloads.

9 | Pilot implementation plan

Week 1: select enterprise-approved asset sources, map evidence provenance and recruit expert reviewers. Week 2: curate and adjudicate a holdout benchmark. Week 3: run AI candidates and rule baselines through a versioned evaluation harness. Week 4: review false negatives, conduct authorized tabletop exercises, publish an auditable results appendix and issue a go/no-go recommendation.

Exit deliverables: dataset card, rubric, model/prompt manifest, confusion matrix, error taxonomy, subgroup analysis, sample audit traces and a reproducibility manifest. Product performance may be published only after actual product execution and independent label review.

10 | Interpretation, limitations and future work

This case illustrates why a security finding is not equivalent to a verified remediation decision. A model can correctly identify a CVE yet incorrectly judge an enterprise host because package backports, deployment modes and provenance matter. Independent evaluation should therefore measure decision correctness and evidence quality, not just fluent vulnerability summaries.

Current limitations: only a synthetic rules exercise is available; public threat intelligence is rapidly evolving; no measured effect on exploit prevention, incident containment, enterprise ROI or AI model reliability can be inferred. Future work should include independent real inventory cases, diverse AI models, prospective shadow-mode trials and controlled expert adjudication.

11 | Public disclosure and claims register

VERIFIED FROM VENDOR / GOVERNMENT SOURCES: two PHP runtime vulnerabilities, CVE identifiers, October 21 release and fixed branches. OBSERVED IN OUR PREVIOUS SYNTHETIC EXERCISE: 240 examples and deterministic rules metrics. PROPOSED, NOT EXECUTED: OpenVals evaluation pipeline, RedShield defensive scenario tests, pilot timeline and release gates.

Safe public claim: “We published a reproducible synthetic study illustrating why vendor backport evidence changes PHP2Shell remediation triage, and propose a framework to independently validate AI-assisted security decisions.” Do not claim “OpenVals detected PHP2Shell,” “RedShield blocked the exploit,” or “100% detection accuracy.”

Appendix A | Operational metric definitions

Precision — TP / (TP + FP); fraction of flagged assets actually requiring action.

Recall — TP / (TP + FN); fraction of assets requiring action correctly flagged.

False-negative rate — FN / (TP + FN); security-critical miss rate.

Abstention rate — Unresolved classifications / all classifications; evaluate separately from accuracy.

Evidence completeness — Share of recommendations with independently verifiable source and package provenance.

Calibration — Agreement between confidence estimates and empirical correctness; use reliability curves and Brier score.

Appendix B | Source register (accessed 23 October 2026)

[1] PHP project security release: https://www.php.net/archive/2026.php#2026-10-21-5

[2] PHP CVE-2026-109618 advisory: https://github.com/php/php-src/security/advisories/GHSA-w8hw-w9fx-j79w

[3] PHP CVE-2026-109619 advisory: https://github.com/php/php-src/security/advisories/GHSA-6wmf-4rv5-p3rp

[4] Canadian Centre for Cyber Security: https://www.cyber.gc.ca/en/alerts-advisories/al26-026-critical-vulnerabilities-impacting-php-cve-2026-109618-cve-2026-109619

[5] Plesk product-specific advisory: https://support.plesk.com/hc/en-us/articles/44397729587351-Vulnerability-CVE-2026-109618-and-CVE-2026-109619-PHP2Shell-Remote-code-execution-in-PHP-shipped-with-Plesk

[6] NCSC-NL incident notes: https://github.com/NCSC-NL/php2shell/blob/main/README.md